

Profiling and Optimizing the TSFM MCP Server

Sally Go
Columbia University
yg3066@columbia.edu

Sam Colman
Columbia University
sc5750@columbia.edu

Byeolah Kwon
Columbia University
bk2833@columbia.edu

Tomas Pasiecznik
Columbia University
tp2758@columbia.edu

Weights & Biases:

<https://wandb.ai/lion-latency/hpml-project-final>

GitHub: <https://github.com/Lion-Latency/AssetOpsBench>

Abstract—Industrial time-series agents often rely on model-backed tools to forecast equipment behavior, adapt models to new sensor data, and detect anomalies in physical assets. In this project, we profile and optimize the Time Series Foundation Model (TSFM) Model Context Protocol (MCP) server used in IBM AssetOpsBench. The main challenge is that tool latency is not determined by model execution alone. Data retrieval, quality filtering, preprocessing, model loading, serialization, and tool orchestration can all contribute to end-to-end runtime.

We build an external benchmarking harness with stage-level instrumentation and Weights & Biases tracking to evaluate forecasting, fine-tuning, time-series anomaly detection (TSAD), and integrated TSAD. We use TinyTimeMixer (TTM) as the primary optimization target and Chronos as a comparison backend. Our optimizations target both preprocessing overhead and inference-side overhead through caching, parallelism, mixed precision, and faster execution paths.

Our results show that the best optimization depends strongly on the workflow. For TTM, Fast Trainer gives the largest forecasting improvement, reducing latency by 33.3%, while inference-side optimizations reduce integrated TSAD latency by 34.6%. Preprocessing parallelism is most effective for fine-tuning, reducing latency by 12.2%. Separately, Chronos is the fastest backend for baseline forecasting, achieving $12.8\times$ lower latency than TTM, but TTM is much faster for fine-tuning. We also find that anomaly detection is the hardest workflow to compare across backends because TTM and Chronos produce different record counts, making direct anomaly-quality comparison unreliable.

Index Terms—time-series foundation models, MCP server, profiling, benchmarking, anomaly detection, model optimization

I. INTRODUCTION

The use of AI agents in industrial settings is becoming increasingly popular. AssetOpsBench provides a unified framework for the development and orchestration of these agents, and the Time Series Foundation Model (TSFM) Model Context Protocol (MCP) Server is a critical component of its functionality. The high-frequency industrial telemetry environments in which it operates contain physical assets such as chillers and turbines. AssetOpsBench agents can use data from these assets for tasks such as forecasting and anomaly detection through MCP tools in the TSFM MCP Server. The various sensors that collect this data require systems with low latency and minimal compute costs. Therefore, optimized workloads are critical for creating safe and viable real-time AI systems. If a dangerous malfunction is occurring, delays

in anomaly detection can be a serious matter. Bottlenecks must be identified to lower latency and compute costs as much as possible, and a naive baseline profiling would not reveal which stage of the pipeline is responsible for the slowdown. We address this by introducing stage-level benchmarking, which provides the level of granularity needed to identify bottlenecks and apply targeted optimizations. Although model inference may be one of the first things that comes to mind when evaluating pipeline latency, other stages in the pipeline should be taken into consideration just as much. Data retrieval, filtering, preprocessing, model loading, serialization, and orchestration can all contribute to slowdowns that affect end-to-end latency.

We aim to optimize the TSFM MCP Server, which is tightly coupled with IBM’s Granite TinyTimeMixer (TTM) as its model, by profiling stage-level latency bottlenecks across forecasting, fine-tuning, anomaly detection, and integrated anomaly detection workloads including data retrieval, preprocessing, model loading, inference/training, and serialization, while introducing a method of decoupling the server from TTM through an interchangeable model interface.

Our profiling and optimizations will include the following 4 workflows from the TSFM MCP Server: forecasting, fine-tuning, TSAD, integrated TSAD. Since the current system does not allow any models other than TTM to be used, we will create an interchangeable model interface that will allow us to use these workflows with Chronos to analyze its performance. Our 5 implemented optimizations will be referenced as follows: cache, parallelism, model cache, fast trainer, and bf16. In addition to this, we test two combination of these optimizations: “combined” (cache + parallelism) and “all inference” (model cache + fast trainer + bf16).

II. LITERATURE REVIEW

A. Time-Series Foundation Models

Time-series foundation models are designed to reuse knowledge across many time-series datasets instead of training a new model from scratch for every task. TinyTimeMixer (TTM) is a compact pretrained model for zero-shot and few-shot multivariate forecasting [1]. Its smaller size makes it a practical backend for repeated benchmarking across forecasting, fine-tuning, and anomaly detection workflows.

Chronos approaches forecasting differently by tokenizing time-series values and modeling them with transformer-based language model architectures [2]. Since Chronos is designed for pretrained and zero-shot forecasting, it gives us a useful comparison point against TTM. In this project, we use both

models under the same benchmark harness to see how model choice affects workflow latency.

B. Industrial Agent Benchmarks and MCP Tool Serving

AssetOpsBench provides a framework for building and evaluating AI agents for industrial asset operations and maintenance [3]. This is the setting our project focuses on: agents that call tools for forecasting, fine-tuning, and anomaly detection over physical assets such as HVAC equipment.

The Model Context Protocol (MCP) is an open standard for connecting AI applications to external tools, data sources, and workflows [4]. In this kind of tool-serving system, latency does not come only from model execution. A single tool call may include data retrieval, preprocessing, model execution, serialization, and server-side orchestration. This is why we profile the full TSFM MCP pipeline instead of measuring inference alone.

C. Profiling and Optimization Techniques

In ML serving pipelines, end-to-end runtime often includes both model-side and system-side costs. For the TSFM MCP server, this includes CPU-side preprocessing, data quality filtering, model loading, inference or training, serialization, and memory usage. Stage-level profiling helps identify which part of the pipeline is actually responsible for latency before applying optimizations.

Common optimizations include caching repeated work, parallelizing CPU-side preprocessing, using mixed precision, and reusing loaded model objects. In our project, preprocessing cache and preprocessing parallelism target data preparation overhead, while model caching, BF16, and fast trainer execution target inference-side or runtime overhead. We evaluate these modes separately because the best optimization depends on the workflow.

III. METHODOLOGY

A. System Overview

We evaluate the TSFM MCP server, which provides time-series foundation model workflows as callable tools. In an agent-based setting, these tools may be called multiple times to forecast equipment behavior, adapt a model to new sensor data, or detect abnormal operating patterns in industrial assets. Because of this, the performance of the full tool pipeline matters, not just the model execution time.

A single TSFM workflow includes several steps around the model itself. Data must be loaded, filtered, preprocessed, passed through the model, and then converted back into outputs that the tool can return. To understand where latency comes from, we profile the pipeline at the stage level rather than treating each workflow as one black-box call.

We focus on four workflows. The *forecasting* workflow predicts future sensor values from historical time-series context. The *fine-tuning* workflow adapts a model to the target industrial dataset. The *TSAD* workflow performs time-series anomaly detection. The *integrated TSAD* workflow combines forecasting with anomaly detection. Together, these workflows

cover both inference-heavy and training-heavy paths in the TSFM server.

B. Dataset

We use real industrial time-series data from IBM AssetOpsBench. For the main benchmark, we use a 100K-row sample to keep experiments reproducible and feasible across multiple workflows, model backends, and optimization modes. Forecasting and fine-tuning are evaluated across five valid equipment groups: CQPA AHU 2B, Chiller 3, Chiller 4, Chiller 6, and Chiller 9. CQPA AHU 1 is excluded because it failed the data quality filter.

For TSAD and integrated TSAD, we restrict evaluation to Chiller 3 only. Full multi-asset anomaly detection triggered data shape, timestamp alignment, and memory-scaling issues. Chiller 3 provides a stable 12-column equipment subset, allowing us to run controlled anomaly detection benchmarks while clearly labeling this as a limitation of the current evaluation.

C. Models

We evaluate two time-series model backends: TinyTimeMixer (TTM) and Chronos. TTM is the default model path in the TSFM MCP server, and we use the `ttm_96_28` checkpoint for the main optimization experiments. This checkpoint is lightweight enough to support repeated benchmarking across forecasting, fine-tuning, TSAD, and integrated TSAD workflows.

Chronos is used as a second backend to evaluate whether the benchmarking harness generalizes beyond the default TTM path. Rather than treating Chronos as an optimization, we use it to study how model choice affects latency under the same workflow-level setup.

To support this comparison, we introduce an interchangeable model interface that decouples the benchmark harness from a single TTM-specific implementation. This allows the same profiling and W&B logging structure to be reused across model backends while keeping model comparison separate from pipeline optimization.

D. Hardware and Software Environment

All experiments are run on Google Cloud Deep Learning VM instances equipped with NVIDIA L4 GPUs with 24 GB of VRAM. The GPU is used for model inference and fine-tuning, while the CPU handles data loading, data quality filtering, preprocessing, workflow orchestration, serialization, and experiment logging.

The software stack includes Python 3.12, PyTorch, Hugging Face Transformers, CUDA, and the TSFM server dependencies. We use Weights & Biases for experiment tracking under the `hpml-project-final` project. Each benchmark run logs its workflow, model backend, optimization mode, dataset scope, latency metrics, memory deltas, and success or failure status.

E. Benchmarking Harness

We use an external benchmarking harness, `benchmark_runner.py`, to run reproducible experiments across workflows, model backends, and optimization modes. Each run corresponds to a specific workflow, model backend, dataset scope, and optimization setting. The harness writes local JSON/CSV summaries and logs metrics to Weights & Biases for comparison across runs.

The harness supports both TTM and Chronos execution paths. This allows us to separate model-backend comparison from pipeline optimization. For each run, we record latency, stage timing, memory deltas, and success or failure status.

F. Profiling Metrics

We collect end-to-end latency, average latency across valid asset groups, and stage-level timing for each benchmark run. The instrumented stages include data retrieval, data quality filtering, preprocessing, model loading, inference or training, and serialization. This breakdown lets us separate core model execution from surrounding pipeline overhead.

We also track CPU/RSS memory deltas and GPU memory deltas when available, since some workflows are limited by memory pressure rather than compute alone. Reliability is recorded through success and failure status for each asset-group run. When available, we log workflow-specific quality metrics for forecasting and fine-tuning so that latency improvements can be interpreted together with output validity.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

All benchmark runs are tracked in Weights & Biases under the `hpml-project-final` project. We tag each run by workflow, model backend, dataset scope, and optimization mode so that matching configurations can be compared consistently.

For forecasting and fine-tuning, we evaluate the 100K-row real-data sample across five valid equipment groups: CQPA AHU 2B, Chiller 3, Chiller 4, Chiller 6, and Chiller 9. For TSAD and integrated TSAD, we report results on the Chiller 3 subset only, since the full multi-asset anomaly detection path triggered shape, timestamp-alignment, and memory-scaling issues.

We run the benchmark harness with both TTM and Chronos. TTM is the primary backend for optimization experiments, while Chronos is used for model-backend comparison under the same workflow setup.

B. Optimization Modes

We evaluate several optimization modes. The preprocessing-focused modes include `cache_only`, which reuses repeated preprocessing work, and `parallelism_only`, which parallelizes CPU-side preprocessing across asset groups. The combined mode applies both preprocessing cache and parallelism.

We also evaluate inference-side modes. The `model_cache` mode avoids repeated model loading,

`fast_trainer` improves the trainer/inference execution path, and `bfl6` enables mixed precision. The `all_inference` mode combines the inference-side optimizations. We report the best observed mode separately for each workflow because the same optimization does not improve all workflows equally.

C. TTM Workflow Optimization Results

We evaluate the effect of each optimization mode on the TTM backend across forecasting, fine-tuning, TSAD, and integrated TSAD. Since each optimization targets a different part of the pipeline, we report the best observed mode separately for each workflow rather than assuming one configuration is optimal for all tasks.

TABLE I
TTM BASELINE VS. BEST OBSERVED OPTIMIZATION.

Workflow	Baseline	Best	Best Mode	Change
Forecasting	46.95s	31.31s	<code>fast_trainer</code>	33.3% faster
Fine-tuning	12.89s	11.32s	<code>parallelism_only</code>	12.2% faster
TSAD	109.79s	107.89s	<code>cache_only</code>	1.7% faster
Integrated TSAD	2335.64s	1527.30s	<code>all_inference</code>	34.6% faster

As shown in Table I, the optimization impact is workflow-dependent. Forecasting improves most with `fast_trainer`, while fine-tuning benefits most from preprocessing parallelism. TSAD shows only a small improvement from caching, while integrated TSAD improves most under `all_inference`, likely because the integrated workflow includes a forecasting step that benefits from inference-side optimization.

These results suggest that the TSFM server does not have a single universal bottleneck. Preprocessing-focused optimizations help some workflows, while inference-side optimizations are more effective when model execution dominates runtime.

D. Stage-Level Profiling and Inference Bottleneck

The TTM optimization results show that preprocessing-focused modes had only a small effect on forecasting, while later inference-side optimization produced a much larger gain. To understand this behavior, we examine the stage-level profile of the TTM forecasting baseline.

For the TTM forecasting baseline, the measured stages include data retrieval, data quality filtering, preprocessing, model loading, inference, and serialization. As shown in Figure 1, inference is the largest measured stage, while preprocessing is relatively small. This explains why preprocessing cache and parallelism alone produced only marginal forecasting gains, and why inference-side optimization was more effective for this workflow.

Model loading is negligible in this path, suggesting that repeated model initialization is not the main bottleneck for TTM forecasting. At the same time, end-to-end latency is larger than the measured stage total, which suggests additional MCP/tool orchestration or per-asset execution overhead outside the directly instrumented stages.

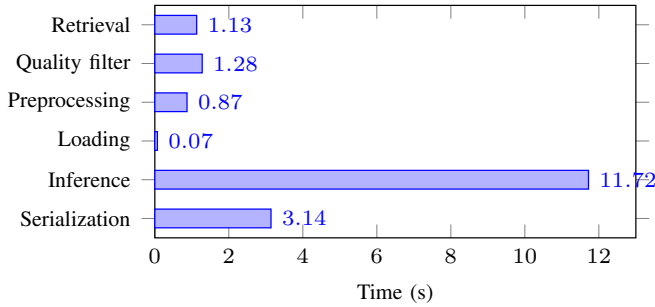


Fig. 1. TTM forecasting baseline stage breakdown.

E. TTM vs. Chronos Model Backend Comparison

We compare TTM and Chronos to understand how model choice affects workflow latency under the same benchmark harness. Table II summarizes the baseline latency for each workflow using both model backends.

TABLE II
TTM VS. CHRONOS BASELINE COMPARISON.

Workflow	TTM	Chronos	Interpretation
Forecasting	46.95s	3.68s	Chronos 12.8 $\times$ faster
Fine-tuning	12.89s	1048.67s	Chronos 81.3 $\times$ slower
TSAD	109.79s	98.03s	Chronos slightly faster*
Integrated TSAD	2335.64s	133.77s	Chronos faster*

Chronos is substantially faster than TTM for forecasting, suggesting that it is well suited for zero-shot forecasting under this benchmark setup. However, Chronos is much slower for fine-tuning, likely because its training path is heavier than TTM’s lightweight adaptation path.

The anomaly detection results require more caution. Although Chronos TSAD and integrated TSAD completed with lower latency, the Chronos and TTM paths produced different record counts in some runs. This suggests differences in preprocessing, windowing, or alignment behavior between the two backends. Because of this, we treat the TSAD and integrated TSAD comparisons as exploratory rather than direct quality comparisons.

F. Anomaly Detection Results and Caveats

The anomaly detection workflows were the hardest part of the benchmark to run reliably. Compared with forecasting and fine-tuning, TSAD and integrated TSAD were more sensitive to input shape, timestamp alignment, and the number of columns included in each run. When we attempted anomaly detection across the full multi-asset dataset, we encountered shape and memory issues. To obtain stable benchmark results, we limited these workflows to the Chiller 3 subset.

Chronos also showed backend-specific behavior in the anomaly detection workflows. Although Chronos TSAD and integrated TSAD completed in some runs, their output record counts differed substantially from the TTM runs. This suggests that the Chronos and TTM paths may handle preprocessing,

windowing, or alignment differently. Because of this, we treat the Chronos anomaly detection results as exploratory and avoid using anomaly counts as a direct quality comparison against TTM.

Overall, these findings show that anomaly detection is not simply another inference workflow. Its performance depends heavily on how the input data is aligned, windowed, and passed into the model backend. Before TSAD performance and quality can be compared fairly across TTM and Chronos, the preprocessing path needs to be standardized.

TABLE III
ANOMALY DETECTION OUTPUT DIFFERENCES BETWEEN TTM AND CHRONOS.

Workflow	TTM Records / Anomalies	Chronos Records / Anomalies
TSAD	10,541 / 507	1,118 / 37
Integrated TSAD	130,247 / 3,657	150 / 8

V. DISCUSSION

A. Interpretation

The impact of optimization was dependent on the workflow because each workflow stressed different pipeline stages and bottlenecks. Fine-tuning benefited most from preprocessing parallelism due to its heavier CPU-side dataset preparation, while forecasting improved only marginally because inference already dominated latency. Integrated TSAD remained the largest bottleneck because it combines forecasting and anomaly detection in a sequential pipeline with additional orchestration overhead. Model comparisons and optimization comparisons should also be separated, since architectural differences and system-level optimizations affect performance independently.

B. Limitations

Our benchmarking of the TSFM MCP Server revealed some limitations of the system. One key limitation is that model size has a large effect on finetuning latency. Table II shows that using Chronos will result in faster forecasting latency but slower finetuning latency. This reveals a trade-off between forecasting latency and fine-tuning latency because using the TTM model will result in slower forecasting latency but faster fine-tuning latency. Another limitation we encountered was that the system is sensitive to data quality. When working with real sensor data, one of the assets would not pass a TSFM data quality filter and therefore not usable. This reveals the importance of proper data collection from the sensors in the industrial settings where AssetOpsBench is used. Finally, we attempted to implement torch.compile with its reduced overhead mode as an optimization to the system but encountered a bug with CUDA graph buffers being reused and overwriting needed data, and lacked the necessary time to fix it.

C. Future Work

Our reproducible benchmarking harness opens up several promising paths for future work. Standardizing TSAD pre-processing across TTM and Chronos would isolate model behavior from pipeline differences in cross-family comparisons, and fixing the Chronos TSAD windowing/alignment quirks (including its hard-coded 15-minute frequency assumption) would unblock direct parity tests. A TSFM_AMP=1 mode using selective torch.autocast would deliver bf16's speedup without its numerical fragility, extending safely to finetuning. Re-enabling torch.compile's mode="reduce-overhead" in the fast-trainer path and rerunning with repeats 5 against a persistent stdio server would amortize cold compile cost and surface its real production payoff. Finally, adding interchangeable_model_interface adapters for other top time-series models would turn the harness from a single-project tool into a reusable cross-family benchmarking framework.

VI. CONCLUSION

AssetOpsBench provides a critical unified framework for AI agents in industrial settings where low latency and compute costs are essential for effective real-time use. Our contributions extend the AssetOpsBench TSFM MCP server by introducing internal profiling instrumentation and an external reproducible benchmarking harness. The workloads we profiled included forecasting, fine-tuning, anomaly detection, and integrated anomaly detection. We implemented five targeted optimizations: a cache optimization, a parallelism optimization, a model cache optimization, a fast trainer optimization, and a bf16 optimization. Finally, we introduced an interchangeable model interface to introduce a method of decoupling the server from TTM and enable benchmarking with other models. We conducted additional benchmarks with Chronos that utilized this interchangeable model interface. As a whole, our team found promising results. We achieved up to 33.3% lower latency with TTM using the fast trainer optimization on forecasting and up to 12.8× faster latency with Chronos using the cache and parallelism optimizations combined on forecasting.

REFERENCES

- [1] V. Ekambaram, A. Jati, P. Dayama, S. Mukherjee, N. H. Nguyen, W. M. Gifford, C. Reddy, and J. Kalagnanam, "Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series," arXiv:2401.03955, 2024.
- [2] A. F. Ansari et al., "Chronos: Learning the Language of Time Series," arXiv:2403.07815, 2024.
- [3] IBM Research, "AssetOpsBench: AI Agents for Industrial Asset Operations and Maintenance," GitHub repository, 2026. [Online]. Available: <https://github.com/IBM/AssetOpsBench>
- [4] Model Context Protocol, "Model Context Protocol Documentation," 2025. [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>